IA Aplicada com LLMs

Aula 13 — MLOps

Roteiro

1. **Por que DevOps não basta** — três eixos de mudança e a degradação sem *commit*
2. **MLOps e a dívida técnica oculta** — a definição, a extensão de DevOps, Sculley et al. (2015) e os portões novos do *pipeline*
3. **Os dez tópicos de MLOps** — do dado ao monitoramento, e o que atravessa o ciclo
4. **Papéis, maturidade e o curso** — os níveis 0, 1 e 2 do Google e onde cada tópico foi ou será tratado

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **justificar** por que sistemas de ML exigem mais que DevOps, pelos três eixos de mudança — código, dados e modelo — e pela degradação sem mudança de código;
- **explicar** a dívida técnica oculta pelos conceitos de Sculley et al. (2015): o código de ML como fração pequena do sistema, CACE, ciclos de *feedback* ocultos, código de cola e *pipeline jungles*;
- **definir** MLOps como extensão de DevOps e **descrever** os dez tópicos, com o mecanismo de cada um;
- **localizar** onde o curso tratou ou tratará cada tópico e **classificar** um sistema nos níveis 0, 1 e 2 do Google.

Parte 1

Por que DevOps não basta

Três eixos de mudança: código, dados e modelo

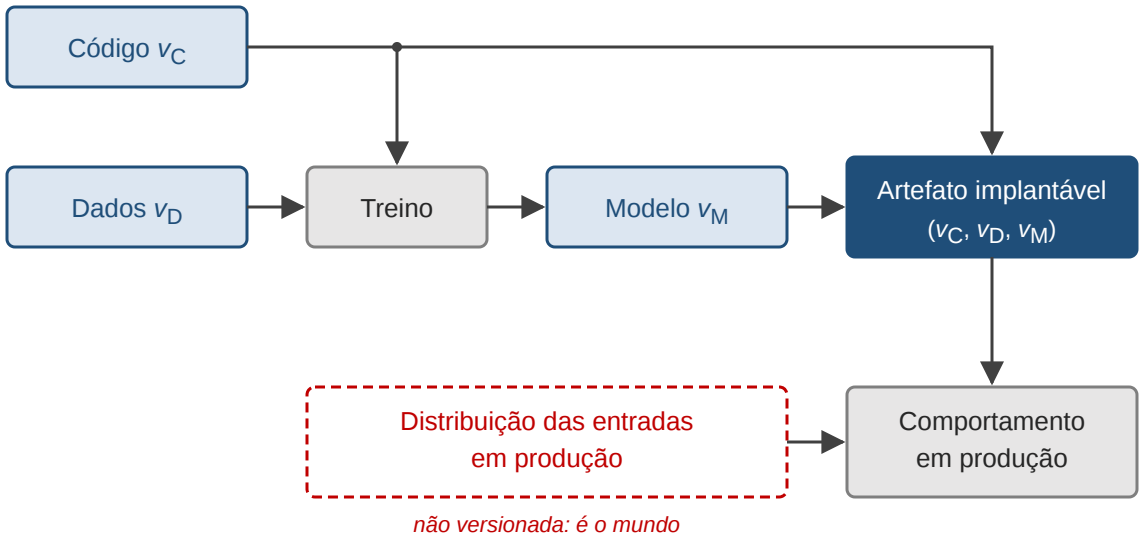
Um **eixo de mudança** é uma dimensão do sistema cuja alteração, isolada, altera o comportamento em produção. Software convencional tem um: o código. Um sistema de ML tem três:

- **código** — preparação de dados, treino, avaliação e inferência;
- **dados** — o conjunto de treino e avaliação, e a distribuição das entradas em produção;
- **modelo** — os parâmetros aprendidos, resultado do código de treino sobre os dados:

$$M = \text{treino}(C_{\text{treino}},\ D,\ h,\ s)$$

com h os hiperparâmetros e s a semente. A identidade do artefato implantável é a tupla (v_C, v_D, v_M), com v_M determinado por (v_C, v_D, h, s) e pelo ambiente. **Reprodutibilidade** é reconstruir M a partir da tupla: exige fixar o código (*commit*), os dados (versão própria) e o ambiente (imagem). Dois modelos do mesmo *commit* com *datasets* diferentes têm o mesmo identificador no repositório e comportamentos distintos.

O controle de versão de código identifica um eixo de três



O código entra duas vezes — no treino e no artefato —; a distribuição das entradas em produção não pertence ao sistema e não é versionada por ninguém.

O modelo degrada sem nenhum *commit*

O modelo aprende uma aproximação de $P(X, Y) = P(Y \mid X)\, P(X)$ observada no treino. Depois do treino, ela pode mudar de duas maneiras:

	O que muda	**Exemplo**
desvio de dados (*data drift*, *covariate shift*)	$P(X)$; $P(Y \mid X)$ se mantém	o modelo de crédito passa a receber outra faixa de renda
desvio de conceito (*concept drift*)	$P(Y \mid X)$: a mesma entrada passa a ter outro rótulo correto	o padrão de transação legítimo vira o de uma fraude nova

- A implantação não mudou, os testes de código passam, latência, taxa de erro e saturação seguem normais: o modelo responde no mesmo tempo e sem exceção — responde errado.
- **Nenhuma métrica de DevOps detecta essa classe de falha.** Daí a camada própria de monitoramento de ML (tópico 6).

Parte 2

MLOps e a dívida técnica oculta

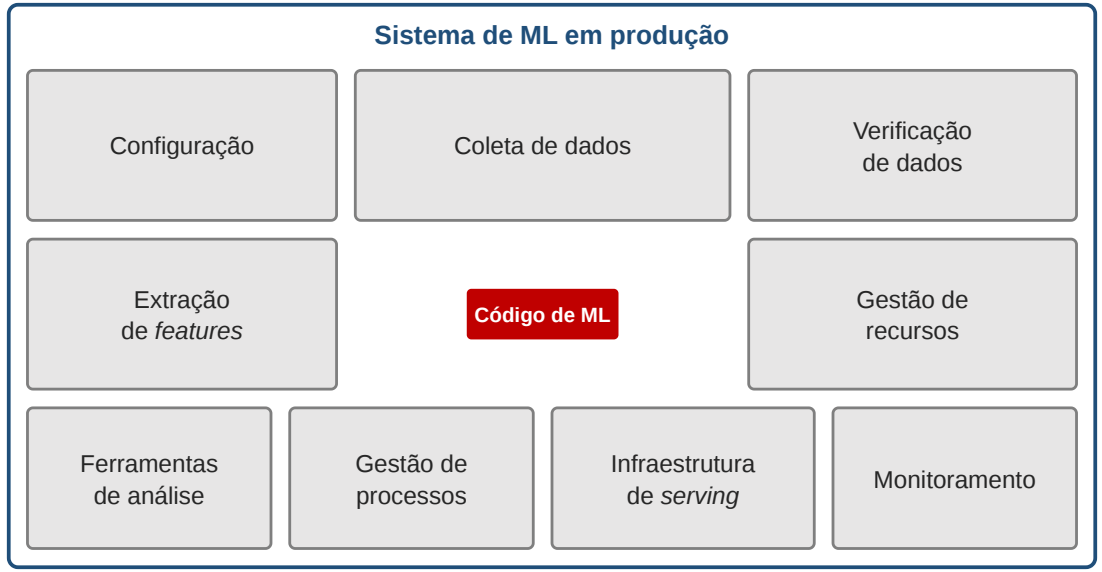
MLOps estende DevOps aos artefatos de ML

MLOps (*machine learning operations*) é o conjunto de práticas, papéis e ferramentas que torna o ciclo de vida de modelos — dados, treino, avaliação, implantação e monitoramento — automatizado, reprodutível e observável (KREUZBERGER; KÜHL; HIRSCHL, 2023). Paradigma, não ferramenta. Os princípios de DevOps permanecem; mudam os artefatos e os papéis.

	DevOps	MLOps
Eixos de mudança	código	código, dados e modelo
Artefato versionado	código, configuração, infraestrutura	os mesmos, mais *datasets*, *features*, modelos e metadados de execução
Integração contínua	testa código	testa código, valida dados e valida modelo
Entrega contínua	implanta a aplicação	implanta o *pipeline* de treino, que produz e implanta o modelo
Gatilho de mudança	*commit*	*commit*, dados novos, alerta de *drift*
Monitoramento	saúde do serviço	saúde do serviço e qualidade das predições
Papéis	desenvolvimento e operação	mais ciência de dados, engenharia de dados e de ML

Contínuo X (AWS): integração e entrega contínuas, mais **treino contínuo** e **monitoramento contínuo**, sem equivalente em DevOps.

O código de ML é uma fração pequena do sistema



Dívida técnica é o custo futuro de manutenção acumulado por decisões que aceleram o presente. Em ML, parte dela fica no **nível do sistema**, fora do código do modelo — e não aparece em revisão de código (SCULLEY et al., 2015).

Três conceitos organizam o resto da dívida

Conceito	Mecanismo	Consequência
CACE (*Changing Anything Changes Everything*)	o peso de uma *feature* depende das demais: mudar a distribuição de x_1, ou acrescentar x_{k+1}, altera todos os pesos no retreino	não há mudança isolada; o encapsulamento não vale para as entradas de um modelo — nem para hiperparâmetros e limiares
ciclos de *feedback* ocultos	a saída do modelo influencia os dados do próximo treino — diretamente, ou por outro sistema no mesmo mercado	o sistema aprende consigo mesmo; nenhum teste isolado acusa, e o efeito é deriva lenta
código de cola e ***pipeline jungles***	adaptação entre os dados do sistema e o pacote de ML; preparação de dados acrescentada por incrementos, sem projeto	o sistema congela em torno do pacote; a *jungle* só se testa ponta a ponta e ninguém a altera com segurança

Outras formas, retomadas nos tópicos: **dependências de dados**, não declaradas nem verificadas por compilador; **consumidores não declarados** da saída do modelo; **dívida de configuração**, quando a configuração é maior e menos testada que o código.

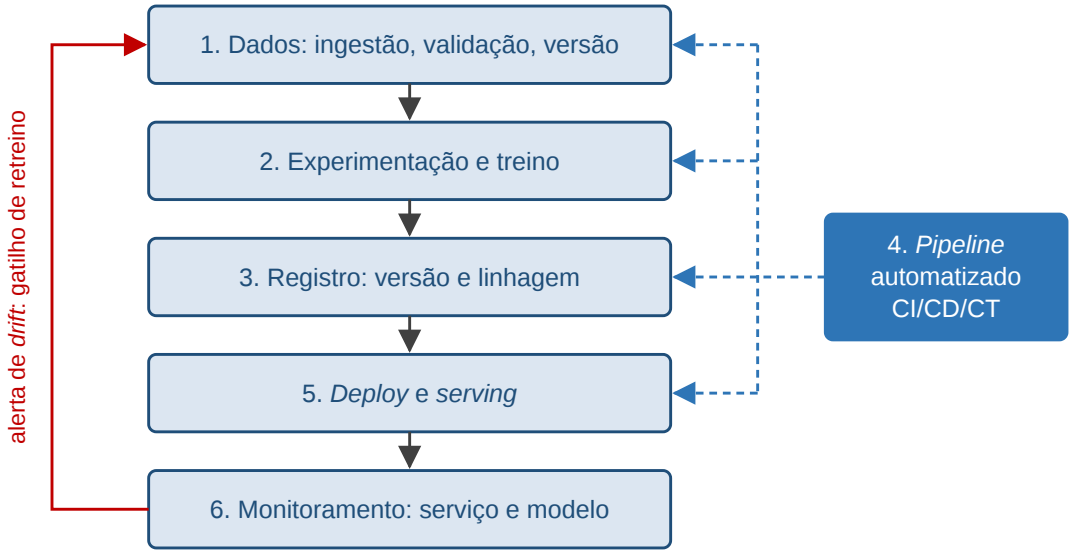
O *pipeline* de ML precisa de portões novos

Em DevOps, os portões operam sobre código: *build*, testes unitários, testes de integração, implantação. Em ML, a mudança entra também por dados e por modelo:

Portão	O que verifica
validar dados antes do treino	*schema*, faixas, ausentes, distribuição contra a referência
avaliar o modelo contra a linha de base	o candidato só avança se não for pior que o modelo em produção, no mesmo conjunto de avaliação
decidir a promoção	registrar o candidato com a linhagem de dados e de código que o produziu, e movê-lo entre estágios

Os portões organizam o ciclo de vida da página seguinte, cujos nós são os tópicos 1 a 6 de MLOps.

O ciclo de vida de MLOps fecha no monitoramento



O alerta de *drift* é o gatilho de retreino. Os tópicos 7 a 10 — testes, infraestrutura, governança e cultura — atravessam o ciclo inteiro e não ocupam um nó dele.

Parte 3

Os dez tópicos de MLOps

Tópicos 1 a 5: do dado ao modelo servido

Tópico	Mecanismo	Exemplos de ferramenta
1. Gestão de dados	*dataset* como artefato: ingestão com origem, validação automática, limpeza como transformação versionada, **versão imutável** por *hash* do conteúdo. A ***feature store*** define cada *feature* uma vez e a serve ao treino e à inferência, contra a **divergência treino-produção** (*training-serving skew*)	DVC, lakeFS; Feast
2. Experimentação	**rastreamento de experimentos**: a cada execução, parâmetros, versões de código e dados, ambiente, métricas e artefatos. Passagem do *notebook* exploratório — ordem livre, estado oculto — para código de treino organizado e testável	MLflow, Weights & Biases
3. Registro de modelos	catálogo de versões com **metadados**, **linhagem** (dados, código e execução que produziram o modelo) e **estágio** (*staging*, produção, arquivado), com promoção controlada	*model registry*
4. *Pipelines* e CI/CD/CT	grafo acíclico dirigido de tarefas; **CI** que valida dados e modelos; **CD** que implanta o *pipeline* de treino; **treino contínuo** por agenda, por dados novos ou por alerta de *drift*, sempre com avaliação antes da promoção	Airflow, Kubeflow Pipelines, Prefect
5. *Deploy* e *serving*	API em tempo real, *batch*, *streaming* ou *edge*; servidor de modelo sobre container e Kubernetes; liberação por ***canary***, ***shadow*** ou **teste A/B**	Docker, Kubernetes; BentoML, Seldon, KServe

Com o CT, o modelo em produção muda sem *commit* e sem pessoa no caminho: o controle de qualidade passa de uma revisão humana para um teste automático. A linhagem torna "predição errada → modelo → execução de treino → *dataset* → lote de ingestão" uma sequência de consultas.

Tópicos 6 a 10: operação e organização em volta do modelo

Tópico	Mecanismo	Exemplos
6. Monitoramento	duas camadas: **serviço** (latência, erro, tráfego, saturação) e **modelo** (desempenho, *data drift*, *concept drift*); o alerta da camada de modelo dispara o retreino	KS, PSI
7. Testes e qualidade	testes de **dados**, de **código**, de **modelo** (linha de base, robustez, **viés por subgrupo**) e de **integração** ponta a ponta	*ML Test Score*
8. Infraestrutura	treino intensivo e intermitente, inferência contínua e sensível a latência: nuvem ou *on-premise*, GPU, infraestrutura como código, instâncias *spot* com *checkpoint*, autoescala, desligamento do ocioso	Terraform
9. Governança	auditoria (a linhagem como prova), explicabilidade, *fairness*, privacidade — LGPD, art. 20: revisão de decisão automatizada —, controle de acesso e documentação pelo ***model card*** (MITCHELL et al., 2019)	*model card*
10. Cultura e maturidade	papéis que DevOps não tem — cientista de dados, engenheiro de dados, engenheiro de ML — e níveis de automação	níveis 0, 1 e 2 (Google)

Sem governança, a automação do tópico 4 promove modelos que ninguém consegue auditar.

Monitoramento de modelo: o rótulo chega tarde, a entrada chega já

O desempenho real só se mede quando o **rótulo verdadeiro** chega — o empréstimo é pago meses depois, a transação é contestada semanas depois. Até lá, o sinal disponível é $P(X)$, observável na hora: um desvio na entrada antecipa a provável queda de desempenho.

O **índice de estabilidade populacional** compara a fração e_i de observações na faixa i da referência com a fração a_i em produção:

$$\mathrm{PSI} = \sum_{i=1}^{k} (a_i - e_i)\, \ln \frac{a_i}{e_i}$$

- $\mathrm{PSI} = 0$ quando as distribuições coincidem; cresce com a divergência. Alternativa: teste de duas amostras de Kolmogorov-Smirnov.
- **Limite**: o sinal antecipado não detecta *concept drift* puro — se $P(X)$ não muda e $P(Y \mid X)$ muda, só o rótulo revela a piora.

ML Test Score: o mínimo, e não a soma

Rubrica de prontidão para produção (BRECK et al., 2017): quatro seções — *features* e dados, desenvolvimento do modelo, infraestrutura de ML, monitoramento —, sete testes cada. Cada teste vale $0{,}5$ se executado manualmente com resultado documentado, e 1 se automatizado e executado repetidamente.

$$\text{Score} = \min_{j \in \{\text{dados, modelo, infra, monitoramento}\}} \sum_{t \in j} p_t, \qquad p_t \in \{0,\ 0{,}5,\ 1\}$$

- O mínimo expressa a tese da rubrica: excelência numa seção não compensa ausência em outra.
- Um sistema com testes de modelo exaustivos e nenhum monitoramento tem pontuação zero — não está pronto.

Parte 4

Papéis, maturidade e o curso

Papéis de MLOps e o muro com mais um lado

Papel	Responsabilidade
Cientista de dados	formular o problema como problema de ML, explorar dados, desenvolver e avaliar modelos
Engenheiro de dados	construir e operar a ingestão e a preparação dos dados
Engenheiro de ML	levar o modelo à produção: *pipelines*, *serving*, monitoramento
DevOps / plataforma	infraestrutura, CI/CD e operação geral
Negócio	definir o objetivo e a métrica de sucesso, e decidir a adoção

Papéis identificados por Kreuzberger, Kühl e Hirschl (2023). O atrito é o muro entre desenvolvimento e operação da nota sobre DevOps, com mais um lado: o cientista de dados é avaliado pela métrica do modelo no *notebook*, o engenheiro pela estabilidade do serviço, e o modelo é "jogado por cima do muro" entre os dois.

Níveis de maturidade 0, 1 e 2 (Google Cloud, 2020)

	Nível 0 — manual	**Nível 1 — *pipeline* de ML automatizado**	**Nível 2 — CI/CD do *pipeline***
Treino	manual, em *notebook*	automático, pelo *pipeline* implantado (CT)	automático (CT)
O que se implanta	o modelo, entregue à operação	o *pipeline* de treino	o *pipeline*, construído e testado por CI/CD
Frequência de liberação	poucas vezes por ano	modelo novo a cada execução do CT	*pipeline* novo a cada mudança de código
Validação	manual	automática, de dados e de modelo	também do código do *pipeline*
Monitoramento de modelo	ausente	presente, com gatilho de retreino	presente, com gatilho de retreino
Componentes	—	*feature store*, metadados, registro de modelos	os do nível 1, mais CI/CD do *pipeline*
Separação entre papéis	total	menor: *pipeline* compartilhado	mínima: experimentação e produção no mesmo código

De 0 a 1 automatiza-se o **treino**; de 1 a 2, a **mudança do *pipeline***. No nível 1, uma alteração na forma de treinar ainda é implantada à mão. A Microsoft propõe outra escala, de cinco níveis (0 a 4).

O sistema do curso não treina, e já pratica MLOps sob outros nomes

Tópico	Onde	O que se fez ou se fará
1. Gestão de dados	*evals*; *embeddings* e RAG	o *golden dataset* revisado e versionado; o índice, que muda as respostas sem mudança de código
2. Experimentação	*prompt engineering*; *evals*	a comparação entre versões de prompt pela suíte, repetida para separar diferença de ruído
3. Registro e linhagem	*prompt engineering*; todo o curso	o versionamento do prompt e o *prompt registry*; o **carimbo de versão** como registro de linhagem
4. *Pipelines* e CI/CD/CT	*evals*	a suíte de regressão como portão antes de cada mudança; sem CT, porque não há treino
5. *Deploy* e *serving*	esta aula, de *script* a imagem	a imagem Docker, a execução em container, o segredo e o estado no container efêmero
6. Monitoramento	observabilidade	telemetria em convenção aberta, painel, alerta, custo e **avaliação online** — o juiz sobre amostra do tráfego
7. Testes e qualidade	*evals*	os quatro níveis de avaliação e o juiz validado contra rótulo humano
8. Infraestrutura	escolha e configuração de modelos; de *script* a imagem	modelo hospedado ou local; a infraestrutura mínima do agente em container
9. Governança e segurança	segurança	o tópico desenvolvido sobre o sistema empacotado
10. Cultura e maturidade	este deck	os papéis e os níveis 0, 1 e 2

O *drift* no sistema do curso tem duas formas: a das perguntas recebidas e a do próprio modelo, quando o provedor atualiza o modelo de um *alias* sem versão ou o deprecia.